

Übungsblatt 1

Einführung & Organisation

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Organisatorisches

Format: Erst die Aufgabe diskutieren, dann zur Lösungsfolie wechseln.

Übung 1.A.1: Kursüberblick

Beantworten Sie folgende Fragen zur Kursorganisation:

1. Welche zwei Prüfungsleistungen gibt es in dieser Lehrveranstaltung?
2. In welchen Teamgrößen wird das Projekt bearbeitet?
3. Wo wird der Projektcode eingecheckt, und warum ist das Tool-Setup bereits zu Beginn wichtig?

Übung 1.A.1: Kursüberblick — Lösung

1. **Klausur** (schriftliche Prüfung) und **Projektarbeit** (Implementierungsprojekt in Teams).
2. Teams von **3 Personen**.
3. Code wird auf dem **FIM GitLab** eingereicht. Frühes Setup ist wichtig, da individuelle Commits für die Benotung herangezogen werden - fehlende Commit-Historie kann nicht nachträglich ergänzt werden.

Übung 1.A.2: Einrichten der Entwicklungsumgebung

Richten Sie Ihre Arbeitsumgebung ein:

1. Erstellen Sie einen FIM-Account und loggen Sie sich auf gitlab.fim.uni-passau.de ein.
2. Erstellen Sie ein neues Repository `wde-exercises` und klonen Sie es lokal.
3. Erstellen Sie die Datei `hello.txt` mit dem Inhalt `Hallo WDE!`, committen Sie sie und pushen Sie den Commit.

Welchen Befehl verwenden Sie für Schritt 3 (alle Git-Kommandos in der richtigen Reihenfolge)?

Übung 1.A.2: Einrichten der Entwicklungsumgebung — Lösung

```
echo "Hallo WDE!" > hello.txt  
git add hello.txt  
git commit -m "Add hello.txt"  
git push origin main
```

Warum diese Reihenfolge? `git add` überführt die Datei in den Staging-Bereich. `git commit` erzeugt den Snapshot. `git push` überträgt den Commit an den Remote-Server.

Grundlagen Web Engineering

Übung 1.B.1: Kategorien von Web-Anwendungen

Ordnen Sie folgende Systeme einer Web-Anwendungskategorie zu (statisch, dynamisch server-seitig, Single-Page Application, Web-Service/API):

System	Kategorie
Universitäts-Stundenplan als HTML-Seite, wöchentlich neu generiert	?
Online-Banking-Portal mit Echtzeit-Kontoübersicht	?
Wetter-REST-API, die JSON zurückgibt	?
Produktkatalog mit Suchfilter ohne Seitenneuladen	?

Begründen Sie je eine Zuordnung kurz.

Übung 1.B.1: Kategorien von Web-Anwendungen — Lösung

System	Kategorie
Stundenplan HTML	Statisch (vorgerendert, kein serverseitiger Code pro Request)
Online-Banking	Dynamisch server-seitig (Session, Authentifizierung, Datenbankabfragen per Request)
Wetter-REST-API	Web-Service/API (kein HTML, maschinenlesbare Daten)
Produktkatalog mit Filter	Single-Page Application (JS übernimmt Routing und Datenabfragen im Browser)

Begründung Wetter-API: Eine API liefert strukturierte Daten (JSON/XML) für Maschinen, nicht für Menschen. Die Darstellung obliegt dem konsumierenden Client.

Übung 1.B.2: Git-Grundlagen — Branching

Sie arbeiten an einem Feature `user-login` in Ihrem Projekt.

1. Welchen Git-Befehl verwenden Sie, um einen neuen Branch `feature/user-login` zu erstellen und direkt zu wechseln?
2. Was ist der Unterschied zwischen `git merge` und `git rebase`?
3. Warum empfiehlt das GitFlow-Prinzip, Arbeiten in Feature-Banches auszulagern statt direkt auf `main` zu arbeiten?

Übung 1.B.2: Git-Grundlagen — Branching — Lösung

1.

```
git checkout -b feature/user-login  
# oder modern:  
git switch -c feature/user-login
```

2. `git merge` erzeugt einen Merge-Commit und bewahrt die vollständige Branch-Historie. `git rebase` schreibt die Commits auf dem Feature-Branch um, sodass sie linear auf dem Zielbranch aufsetzen - sauberere Historie, aber verändert bestehende Commits.
3. Feature-Branches isolieren Arbeiten voneinander: `main` bleibt jederzeit deploybar, parallele Features stören sich nicht, und Code-Reviews über Pull/Merge Requests sind vor dem Merge möglich.

Was ist Web Engineering?

Übung 1.C.1: Web Engineering — Ziele und Prinzipien

Web Engineering ist eine systematische Disziplin zur Entwicklung von Web-Anwendungen.

1. Formulieren Sie in eigenen Worten: Was unterscheidet **Web Engineering** von klassischer Software-Entwicklung?
2. Nennen Sie drei typische Qualitätsziele, die Web-Anwendungen im Gegensatz zu Desktop-Software besonders stark betonen.
3. Ein Unternehmen entwickelt seine Web-Anwendung ohne Versionskontrolle, ohne Tests und ohne dokumentierte Anforderungen. Welche konkreten Probleme entstehen kurz- und langfristig?

Übung 1.C.1: Web Engineering — Ziele und Prinzipien — Lösung

1. **Web Engineering** behandelt die Besonderheiten von Web-Anwendungen: heterogene Clients (Browser, Mobilgeräte), das zustandslose HTTP-Protokoll, öffentliche Erreichbarkeit, schnelle Releasezyklen, verteilte Teams und hohe Nutzerzahlen. Klassische SE optimiert oft für einen definierten Client-Typ und ein kontrolliertes Deployment-Umfeld.
2. Typische Web-Qualitätsziele:
 - **Skalierbarkeit:** Tausende bis Millionen gleichzeitiger Nutzer
 - **Accessibility/Usability:** Unterschiedliche Endgeräte, Screenreader, Barrierefreiheit
 - **Performance/Ladezeit:** Nutzer verlassen Seiten nach mehr als 3 Sekunden Ladezeit
3. **Kurzfristig:** Kein Nachvollziehen von Änderungen, Überschreiben von Kollegenarbeit, keine automatisierte Qualitätssicherung. **Langfristig:** Technische Schulden akkumulieren, Regression bei Änderungen, keine Basis für Compliance oder Audits, schwieriges Onboarding neuer Entwickler.

Übung 1.C.2: Evolution des Web — 1.0, 2.0, 3.0

Das Web hat sich in Generationen entwickelt:

Merkmal	Web 1.0	Web 2.0	Web 3.0
Inhaltsproduktion	?	?	?
Nutzerinteraktion	?	?	?
Typische Technologien	?	?	?
Beispiel-Anwendung	?	?	?

Füllen Sie die Tabelle aus und erklären Sie: Welches Web-Paradigma dominiert heute in Unternehmensanwendungen?

Übung 1.C.2: Evolution des Web — 1.0, 2.0, 3.0 — Lösung

Merkmal	Web 1.0	Web 2.0	Web 3.0
Inhaltsproduktion	Nur Betreiber	Nutzer generieren Inhalte (UGC)	Dezentralisiert, maschinenlesbar
Nutzerinteraktion	Passiv (Lesen)	Aktiv (Kommentieren, Teilen)	Semantisch, personalisiert, KI-gestützt
Typische Technologien	Statisches HTML, FTP	AJAX, RSS, Web-APIs, Blogs	Linked Data, RDF, KI/ML-Integration
Beispiel-Anwendung	Unternehmens-Broschüre-Website	Facebook, Wikipedia, YouTube	Semantische Suchmaschinen, KI-Assistenten

Heute dominiert das Web-2.0-Paradigma in Unternehmensanwendungen: interaktive Single-Page Applications, REST-APIs, nutzererzeugte Daten. Web-3.0-Elemente (semantische Metadaten, KI-Empfehlungen) werden zunehmend integriert.